Homework 09

Rahaf Alabdullah (rahafaa2)

Question 1: Choosing an SVM kernel

```
import numpy as np
from pathlib import Path
import pandas as pd
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

data_dir = Path(__file__).parent / "data"
heart = pd.read_csv(data_dir / "saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd").to_numpy()
y = heart["chd"].to_numpy()
n, p = X.shape

rng = np.random.default_rng(43235)
folds = np.empty(n, dtype=int)
shuffled_indices = rng.permutation(n)
folds[shuffled_indices] = np.arange(n) % 5

C_linear = [0.1, 1, 10]
linear_candidates = [{"kernel": "linear", "C": C, "tol": 1e-7} for C in C_linear]

gamma_poly = 1 / p
C_poly = [0.1, 1]
poly_candidates = [{"kernel": "poly", "degree": degree, "C": C, "gamma": gamma_poly, "coef0": 0, "tol": 1e-7} for degree in [2, 3, 4] for
C in C_poly]

C_radial = [0.1, 1, 10]
gamma_radial = [0.01, 0.1, 1]
radial_candidates = [{"kernel": "rbf", "C": C, "gamma": gamma, "tol": 1e-7} for C in C_radial for gamma in gamma_radial]


all_candidates = linear_candidates + poly_candidates + radial_candidates
cv_errors = [[] for _ in all_candidates]

for k in range(5):
    train = folds != k
    test = folds == k
    X_train, y_train = X[train], y[train]
    X_test, y_test = X[test], y[test]

    scaler = StandardScaler().fit(X_train)
    X_train_scaled = scaler.transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    for i, candidate in enumerate(all_candidates):
        svc = SVC(**candidate).fit(X_train_scaled, y_train)
        predictions = svc.predict(X_test_scaled)
        misclassification_rate = np.mean(predictions != y_test)
        cv_errors[i].append(misclassification_rate)

print(pd.DataFrame([{**candidate, "mean_cv_error": np.mean(errors), "se_cv_error": np.std(errors, ddof=1) / np.sqrt(len(errors))} for
candidate, errors in zip(all_candidates, cv_errors)]).sort_values("mean_cv_error").round(3).to_string(index=False))
```

Candidate settings and mean 5-fold CV misclassification rates (sorted by CV error; tol = 1e-7 for all fits):

Kernel	C	Degree	γ	coef0	Mean CV error	SE
Radial	10	–	0.01	–	0.270	0.024
Radial	1	–	0.01	–	0.279	0.028
Linear	0.1	–	–	–	0.279	0.020
Linear	10	–	–	–	0.281	0.014
Linear	1	–	–	–	0.281	0.015
Radial	1	–	0.1	–	0.296	0.021
Polynomial	1	3	1/9	0	0.301	0.036
Radial	10	–	0.1	–	0.307	0.029
Polynomial	1	2	1/9	0	0.322	0.023
Polynomial	0.1	4	1/9	0	0.325	0.019
Polynomial	0.1	3	1/9	0	0.331	0.016
Polynomial	1	4	1/9	0	0.335	0.022
Polynomial	0.1	2	1/9	0	0.338	0.018
Radial	0.1	–	0.1	–	0.346	0.020
Radial	0.1	–	0.01	–	0.346	0.020
Radial	0.1	–	1	–	0.346	0.020
Radial	1	–	1	–	0.353	0.027
Radial	10	–	1	–	0.379	0.020

Candidate settings (18 in total, all evaluated on the same five folds, made with seed 43235; predictors standardized within each fold using only the fitting observations):

- **Linear (3):** C in {0.1, 1, 10}; everything else held fixed.
- **Polynomial (6):** degree in {2, 3, 4} x C in {0.1, 1}; held fixed: gamma = 1/p = 1/9 and coef0 = 0.
- **Radial (9):** C in {0.1, 1, 10} x gamma in {0.01, 0.1, 1}; everything else held fixed.
- **All kernels:** tol = 1e-7; all other SVC options at sklearn defaults.

Preferred model: radial kernel with C = 10 and gamma = 0.01.

Justification: this setting has the lowest mean CV misclassification rate (0.270, SE 0.024). The linear kernel with C = 0.1 is a close second (0.279, SE 0.020). The difference between them (about 0.009) is smaller than one standard error, so the CV results do not clearly separate the two, and the simpler linear SVM would be a reasonable alternative. Polynomial kernels did worse (0.301 to 0.338), and radial fits with large gamma (gamma = 1) or small C (C = 0.1) did no better than the 0.346 error from always predicting the majority class (chd = 0). The small gamma that worked best makes the radial boundary smooth and close to linear, which agrees with the linear kernel performing almost as well.

Question 2: SVM analysis skill

The one-page, 11-point report produced by invoking the `svm-analysis` skill follows on the next page. The complete `SKILL.md` is appended after the report.

SVM analysis of saheart.csv: predicting chd

n = 462, p = 9 (famhist: 1 = Absent, 2 = Present), seed 43235, 5 folds. No missing values; chd = 0/1: 302/160 (majority-class error 0.346).

Methods. n ≫ p, predictors are numeric on very different scales (standardized within each training fold), and single-predictor class separation is modest (largest standardized mean difference 0.78, age), so we compared linear, low-degree polynomial (degree 2–3, γ = 1/p, coef0 = 1) and RBF kernels, spanning linear to smooth nonlinear boundaries. One random 5-fold split (seed 43235) was shared by all 29 settings: C ∈ {0.01, ..., 100} (linear), degree × C ∈ {0.01, ..., 10} (polynomial), C ∈ {0.1, ..., 100} × γ ∈ {0.001, ..., 1} (RBF), factor-10 steps. The lowest mean CV misclassification rate was selected and evaluated by accuracy, precision, recall and specificity of its out-of-fold predictions.

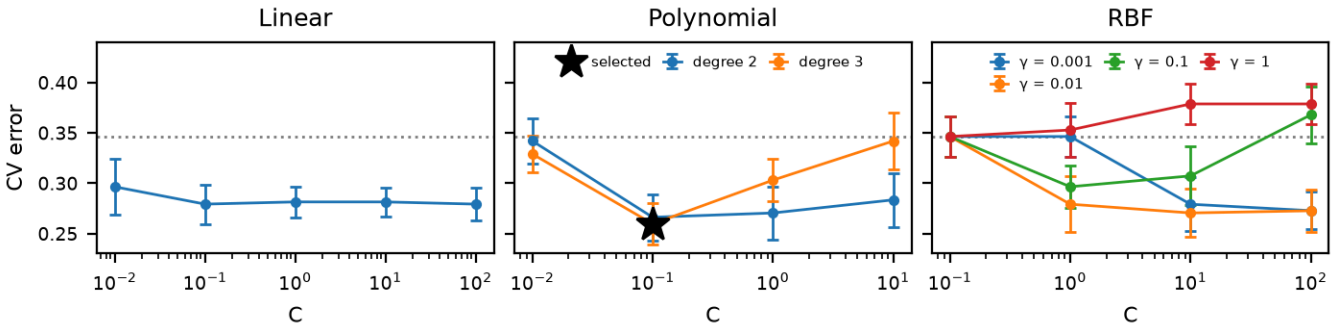


Figure 1: CV misclassification (mean ± 1 SE) against C. Dotted line: majority-class error. Star: selected model.

Model	Best setting	CV error	SE
Linear	C = 0.1	0.279	0.020
Polynomial	**degree 3, C = 0.1**	**0.260**	**0.021**
RBF	C = 10, γ = 0.01	0.270	0.024

Results. The selected model is the **polynomial kernel with degree 3, C = 0.1**, with CV error 0.260 (SE 0.021) vs. 0.346 for always predicting no CHD. Out-of-fold accuracy is 0.740, precision 0.717, recall 0.412, specificity 0.914 (66/160 cases found): a CHD flag is usually right, but most cases are missed.

Findings and limitations.

- Tuned kernels perform alike: best errors (linear 0.279, RBF 0.270, polynomial 0.260) differ by ≤ 0.020 (≈ 1 SE); 10 of 29 settings are within 1 SE of the minimum, so a linear SVM is a defensible simpler choice.
- Tuning matters more than kernel: extreme C or large γ gives errors near or above the majority-class error; good RBF fits use small γ (near-linear boundaries).
- The chosen C is interior to its grid, but grids are coarse and selection and reporting use the same folds, so 0.260 is optimistic; nested CV would be unbiased.
- One 5-fold split with SEs ≈ 0.02: another seed may reorder kernels. Linear C = 10 and Linear C = 100 hit the solver iteration limit; neither was selected.
- The default rule (sign of the decision function) gives low recall; class weights or a shifted threshold would trade precision for sensitivity.

Appendix: SKILL.md

Complete contents of `.claude/skills/svm-analysis/SKILL.md`:

```
---
name: svm-analysis
description: Summarize SVM analysis results into a one-page report.
disable-model-invocation: true
---
# SVM Analysis Report
Run this skill only when the user invokes it explicitly. The user supplies the
dataset, response, predictors (with any coding), a seed and
output folder. If any of these are missing, ask before starting. Never assume
dataset-specific facts; base every statement on the results you compute.

## Tools
Python with numpy, pandas, sklearn.preprocessing, sklearn.svm. Write one script that reproduces every number, figure and table in
the report.

## Analysis rules
1. **Load and check.** Report n, number of predictors, class counts, and any
   missing values. Do not drop or recode variables beyond what the user asks. Judge whether the data are suitable for an SVM analysis
2. **choosing candidate kernels.** Explore the data. Choose 5 folds, and a kernel (linear, polynomial, RBF) from each of the three families
that seems appropriate for the data. Justify your choice based on the characteristics of the data and the problem at hand.
3. **Hyperparameter tuning.** identify the hyperparameters of the chosen kernel that need to be tuned (e.g., C, degree, gamma). Use cross-
validation to find the optimal values for these hyperparameters. Report the range of values tested and the corresponding performance metrics.
4. **Model evaluation.** Evaluate the performance of the SVM model with the chosen kernel and optimal hyperparameters using appropriate
metrics (e.g., accuracy, precision). Report the results and provide a brief interpretation of the model's performance.

## Report
Write a Quarto document (`.qmd`) that renders to a PDF of **at most one
printed page** (letter size, 11pt, 0.75in margins). Hide code; show only
results. Render it and confirm the page count; shorten text, not results, if it
is too long. Sections, in order:
1. **Title** naming the dataset and response, plus one line with n, p, the
   seed and fold number.
2. **Methods** (3-4 sentences): candidates, fold selection, evaluation methods, and tuning    hyperparameters.
3. **Figure**: CV misclassification rate (mean +/- 1 SE) for each candidate
   against its tuning setting. Mark the selected model.
4. **Table**: one row per candidate at its best setting: model,
   setting, CV error, SE. Bold the selected model. Rates to 3 decimals.
5. **Results** (2-3 sentences): selected model and setting, its
   misclassification rate, and intrepretation.
6. **Findings and limitations** (3-5 bullets): what the comparison suggests
   about the SVM kernel, hyperparameter tuning, and model performance.
```